



12.2. Operations on Heaps

"Keep the strongest element at the top, no matter what happens below."

Previous Section:

 [12.1. Heaps](#)

Contents:

[1. Basic Operations](#)

[1.1. Traversal](#)

[1.2. Insertion](#)

[1.3. Deletion of the Root Node](#)

[1.4. Time Complexity of Heap Operations](#)

[1.5. Deletion of Any Node in a Heap](#)

[2. Implementing Heaps with Python](#)

[2.1. Max Heap Class](#)

[2.2. Detecting a Leaf Node](#)

[2.3. Swapping Two Nodes](#)

[2.4. Insertion](#)

[2.5. Displaying the Heap](#)

[2.6. Accessing the Root Element](#)

[2.7. Heapify Operation](#)

[2.8. Deletion](#)

[2.9. Converting a Max Heap into a Min Heap](#)

[2.10. Converting an Entire Heap](#)

[Summary](#)

[References](#)

Building a heap is only the beginning. The real power of a heap comes from its ability to continuously maintain order while elements are inserted and removed.

Imagine a hospital using a Priority Queue implemented with a heap. Patients arrive. Patients leave. Emergency cases appear unexpectedly.

Despite all these changes, the heap must always guarantee one thing:

- Min Heap → Smallest value at the root
- Max Heap → Largest value at the root

How does it accomplish this? Through a special process called **Heapify**.

Whenever a modification causes the heap property to be violated, the heap reorganizes itself until order is restored.

In this section, we will study the fundamental operations performed on heaps and learn how heapify keeps the structure valid.

1. Basic Operations

Like other tree structures, heaps support three primary operations: Traversal, Insertion, and Deletion.

However, unlike ordinary trees, heaps must always preserve the heap property after modifications.

1.1. Traversal

Since a heap is still a binary tree, it can be traversed using the standard tree traversal methods. These include: In-order Traversal; Pre-order Traversal; Post-order Traversal; Breadth-First Traversal

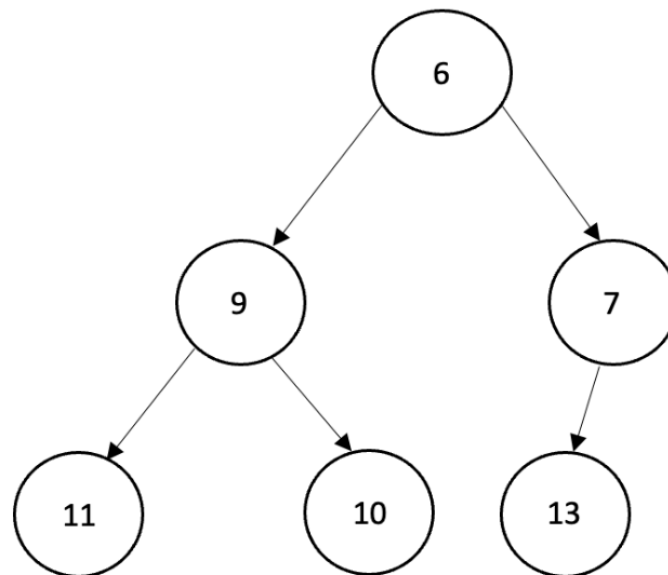
Different traversal methods will visit nodes in different orders, just like any other binary tree.

Unlike Binary Search Trees, traversing a heap does **not** automatically produce a sorted sequence. The heap only guarantees that parent-child relationships satisfy the heap property.

1.2. Insertion

Insertion begins similarly to a Complete Binary Tree. The new node is always inserted into the next available position following level-order insertion.

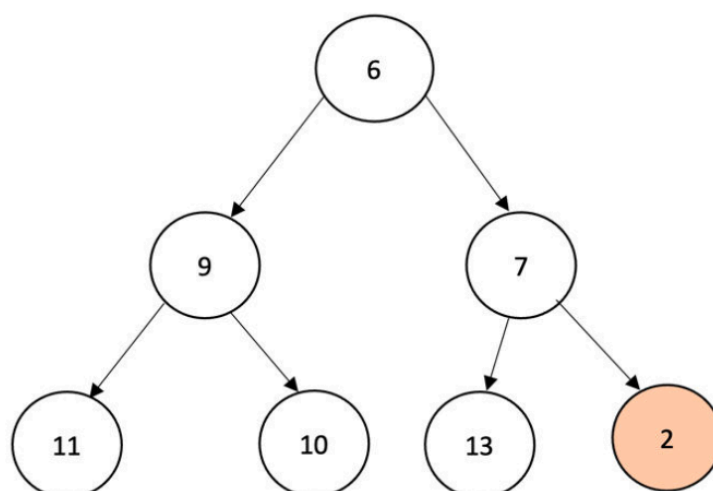
Suppose we have the following Min Heap:



Array representation: [6, 9, 7, 11, 10, 13]

- **Step 1: Insert the New Node**

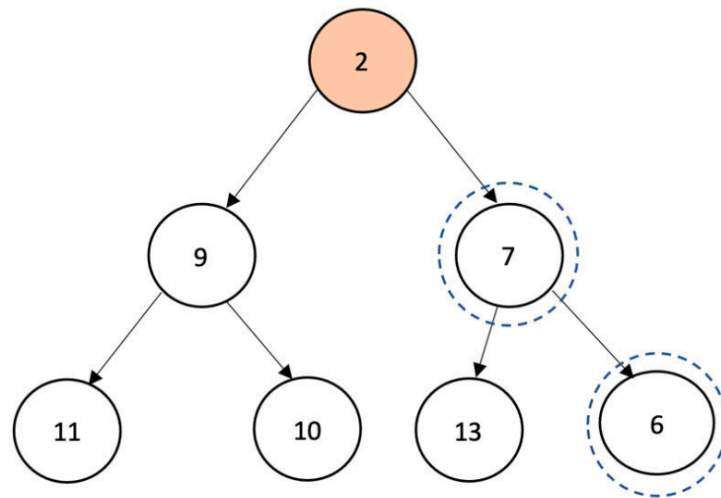
Insert node 2. The next available position is the right child of 7.



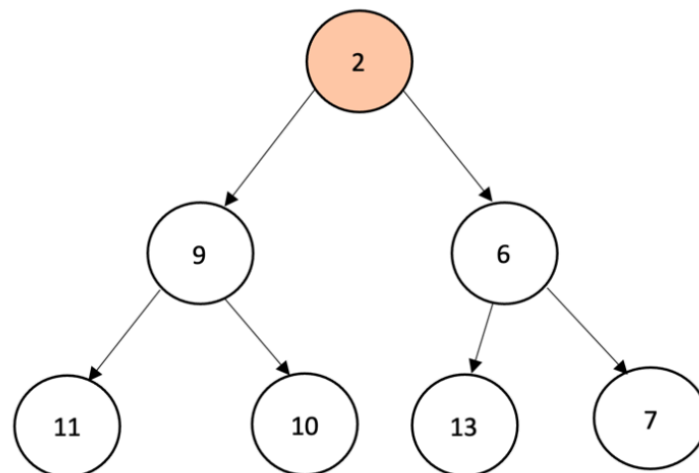
Immediately, a problem appears: $7 > 2$. This violates the Min Heap property.

- **Step 2: Heapify Up**

Swap node 2 with its parent.



Another violation still exists: $6 > 2$. Heapify continues. Swap again.



Now the heap property is satisfied. Insertion is complete.

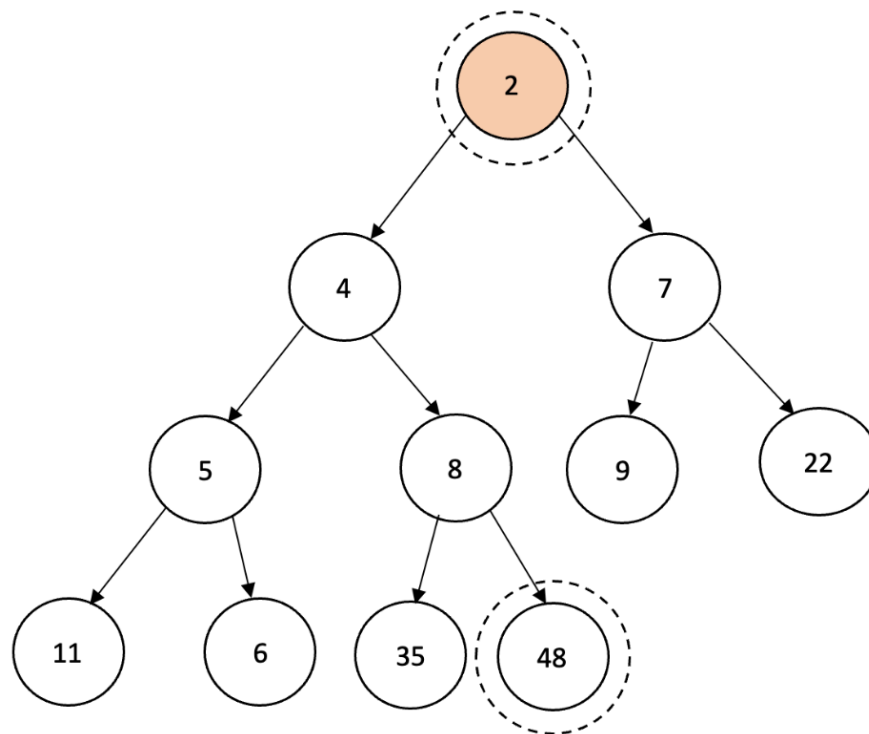
This process is called **Heapify Up** (or Bubble Up, or Percolate Up). All three terms describe the same idea. The new node keeps moving upward until the heap property is restored.

1.3. Deletion of the Root Node

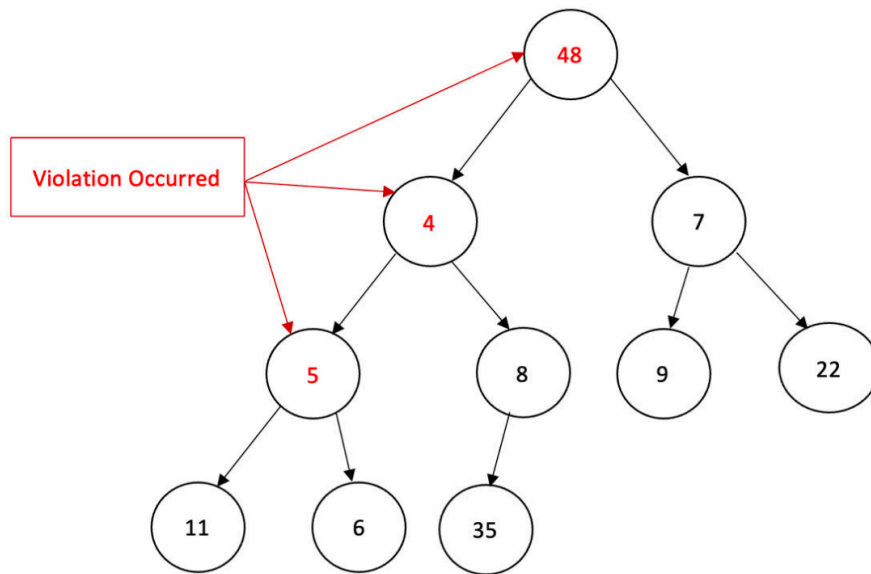
Unlike ordinary trees, heaps primarily remove elements from the root. This behavior is what makes heaps useful for Priority Queues.

- **Step 1: Remove the Root**

Delete node 2. A hole now exists at the root.



Array representation for this heap: [2, 4, 7, 5, 8, 9, 22, 11, 48, 35]

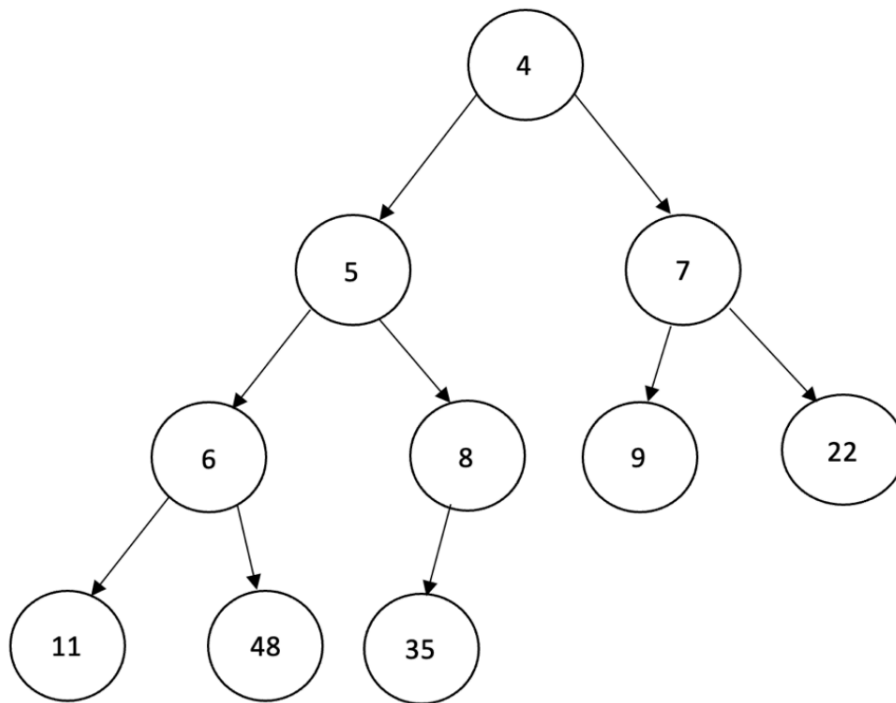


The current tree violates the Min Heap property → We need to heapify

Heapify to restore the Min Heap property:

- **Step 2 - Move the Last Node:** Move the last node (35) to the root. The heap property is now violated: $35 > 4$ and $35 > 7$. For a Min Heap, the root should be the smallest value.
- **Step 3 - Heapify Down:** Compare 35 with its children. The smaller child is 4. So swap.
 - Violation still exists: $35 > 5$. Swap again.
- Now the heap property should be restored.

This process is called Heapify Down (or Bubble Down, or Percolate Down)



If we are going for Max Heap

- **Step 2 - Move the Last Node:** Move the last node (35) to the root. The heap property is now violated: $35 < 48$ (its left child). For a Max Heap, the root should contain the **largest** value.
- **Step 3 - Heapify Down:** Compare 35 with its children. The **larger child is 48**, so swap them.
 - Continue comparing 35 with its new children. If one of its children is still larger than 35, swap again.
 - Repeat this process until **35 is greater than or equal to both of its children**, or it becomes a leaf node.
- Now the Max Heap property has been restored.

1.4. Time Complexity of Heap Operations

One remarkable property of heaps is their efficiency. For a heap containing n elements: $Height \approx \log_2(n)$

Since heapify only travels along a single path from root to leaf (or leaf to root), the number of swaps is limited by the height of the tree. Therefore:

Operation	Complexity
Insert	$O(\log n)$
Delete Root	$O(\log n)$
Peek Root	$O(1)$

Heapify never explores the entire tree. It only moves along one branch. This is why heaps remain efficient even for very large datasets.

1.5. Deletion of Any Node in a Heap

Earlier, we mentioned that heaps usually remove the root node. But what if we want to delete a node somewhere else?

For example: Delete node 8, or node 25, or node 42. Is it possible? Yes. However, additional work may be required to restore the heap property.

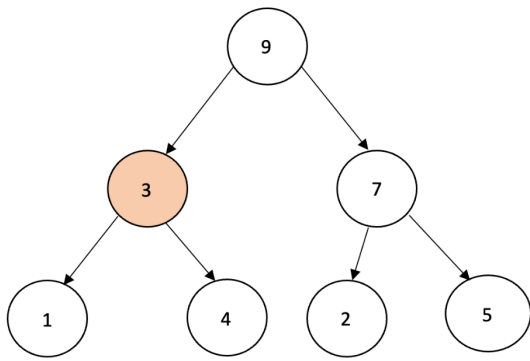
General Procedure

To delete a node at any position:

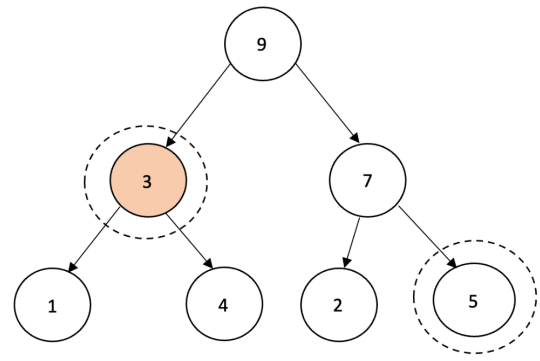
1. Select the node.
2. Swap it with the last node.
3. Remove the last node.
4. Heapify if necessary.

Example 1 – No Heapify Required

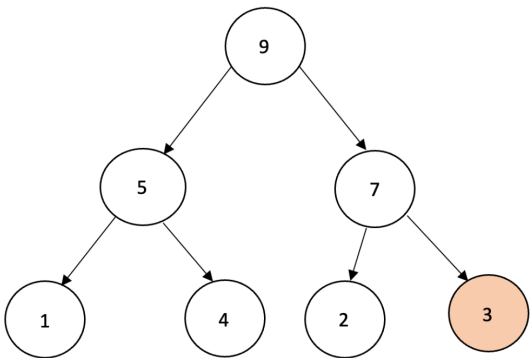
Consider this Max Heap:



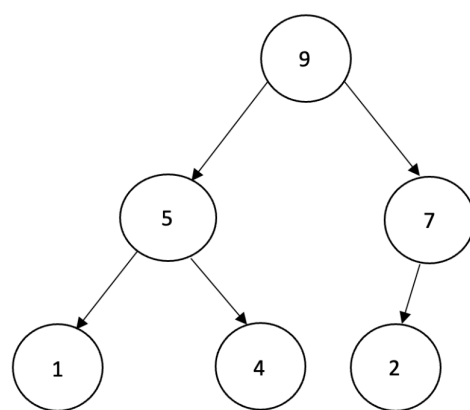
Key Node to remove is 3



Swap Node 3 and Node 5



Node 3 is now at the leaf

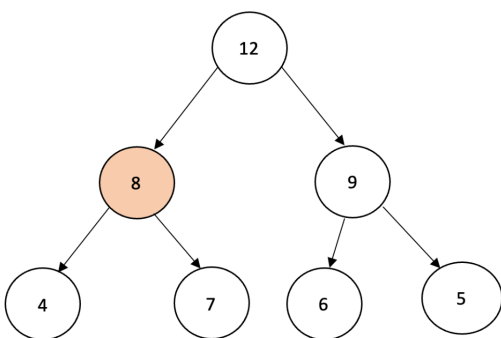


Delete Node 3

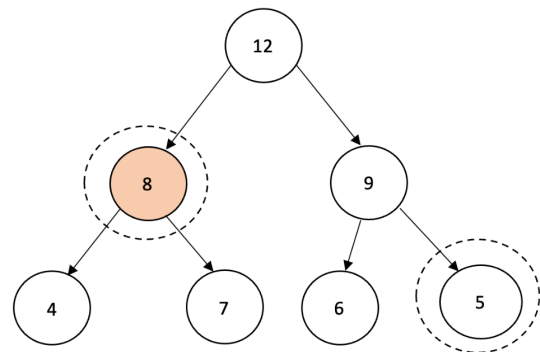
Observe: $9 > 5 > 1, 4$. The heap property remains valid. No heapify is required.

Example 2 – Heapify Required

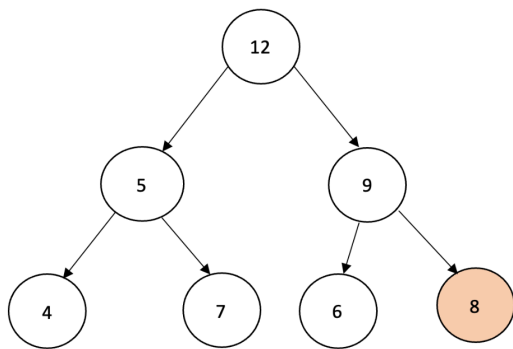
Consider the following Max Heap:



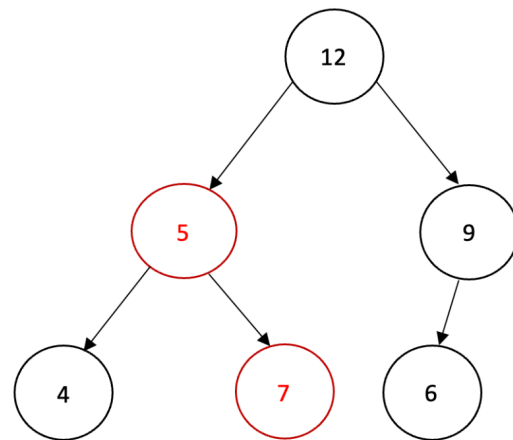
Key Node to remove is 8



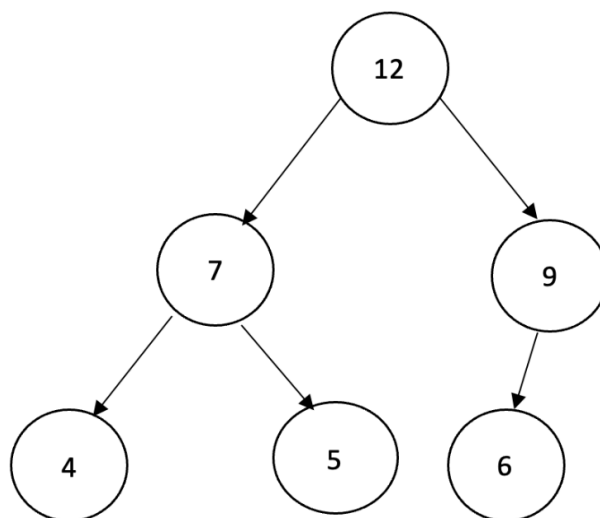
Swap Node 5 and Node 8



Node 8 is deleted but the Max heap property is violated



Heapify Node 5 and Node 7



Now the Max heap property is restored

Why Heapify Matters

Heapify is the mechanism that gives heaps their power. Without heapify, insertions, deletions, or modifications would quickly destroy the heap structure. With heapify, the heap property is always maintained

No matter how many elements are inserted or removed, the heap continuously reorganizes itself to ensure that the root always contains the minimum value (Min Heap) or maximum value (Max Heap).

2. Implementing Heaps with Python

Understanding heap operations conceptually is important, but eventually we need a way to implement them in code. Fortunately, heaps can be represented efficiently using arrays, making them relatively straightforward to implement in Python.

In this section, we examine a simple implementation of a **Max Heap** and then explore how it can be modified to create a **Min Heap**.

2.1. Max Heap Class

The following class implements a Max Heap using an array.

```
class MaxHeap:
    def __init__(self, maxsize=100):
        self.maxsize = maxsize
        self.heap = [0] * self.maxsize
        self.last = -1
        self.peak = 0

    @staticmethod
    def parent(pos):
        return (pos - 1) // 2

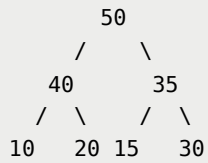
    @staticmethod
    def left_child(pos):
        return 2 * pos + 1

    @staticmethod
    def right_child(pos):
        return 2 * pos + 2
```

The heap is stored entirely inside an array.

```
Index:  0  1  2  3  4  5  6
Value: [50, 40, 35, 10, 20, 15, 30]
```

Visualization:



The helper functions allow us to navigate between parent and child nodes without storing explicit links.

Function	Purpose
<code>parent(i)</code>	Returns parent index
<code>left_child(i)</code>	Returns left child index
<code>right_child(i)</code>	Returns right child index

For example:

```

Index = 1

Left Child = 2(1)+1 = 3
Right Child = 2(1)+2 = 4
  
```

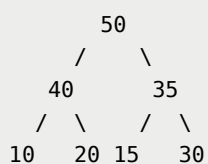
2.2. Detecting a Leaf Node

A leaf node has no children.

```

def is_leaf(self, pos):
    return self.left_child(pos) > self.last
  
```

Visualization:



The nodes: `10 20 15 30` are leaf nodes because they have no children.

This function is useful during heapify operations because leaf nodes already satisfy the heap property.

2.3. Swapping Two Nodes

```
def swap(self, pos1, pos2):  
    self.heap[pos1], self.heap[pos2] = \  
        self.heap[pos2], self.heap[pos1]
```

Heapify relies heavily on swapping. Before: `[20, 50]` → After swapping: `[50, 20]`

2.4. Insertion

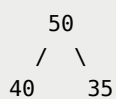
Insertion follows two steps:

- Step 1: Insert at the Next Available Position

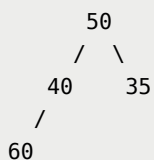
```
self.last += 1  
self.heap[self.last] = value
```

Example:

Current Heap



Insert: 60



The heap property is now violated.

- Step 2: Heapify Up. The newly inserted node moves upward until the Max Heap property is restored.

```

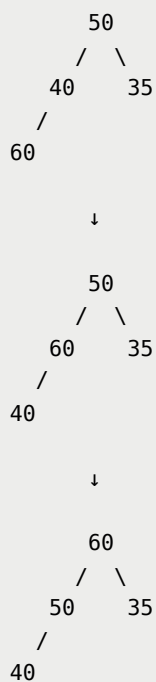
while (self.parent(current) >= 0 and
      self.heap[current] >
      self.heap[self.parent(current)]):

    self.swap(current,
                self.parent(current))

```

Visualization:

Insert 60



The largest element eventually reaches the root.

2.5. Displaying the Heap

```

def display(self):
    for i in range(self.last + 1):
        print(self.heap[i], end=" ")

```

Output: 60 50 35 40

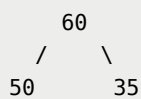
This simply displays the internal array representation of the heap.

2.6. Accessing the Root Element

```
def get_peak(self):  
    if self.last >= 0:  
        return self.heap[self.peak]
```

The root node always stores the maximum value in a Max Heap.

Visualization:



Therefore: `heap.get_peak()` returns 60 without removing it.

Time Complexity: $O(1)$

2.7. Heapify Operation

The heart of the heap implementation is the **heapify** process.

```
def max_heapify(self, pos):  
    if not self.is_leaf(pos):  
        # In Max Heap, the child node is smaller than the  
        # father node  
        if (self.heap[pos] < self.heap[self.left_child(pos)] or  
            self.heap[pos] < self.heap[self.right_child(pos)]):  
            # Heapify the left node and recursive left child tree  
            if self.heap[self.left_child(pos)] > self.heap[self.right_child(pos)]:  
                self.swap(pos, self.left_child(pos))  
                self.max_heapify(self.left_child(pos))
```

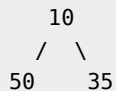
```

else: # Heapify the right node
    self.swap(pos, self.right_child(pos))
    self.max_heapify(self.right_child(pos))

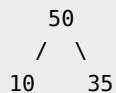
```

Whenever the heap property is violated, heapify restores it.

Consider:



This is not a valid Max Heap because $10 < 50$. Heapify compares the parent with its children and swaps with the larger child.



The process continues recursively until the subtree becomes a valid heap.

2.8. Deletion

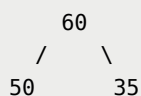
Deletion always begins at the root.

```

def delete(self):
    size = self.last + 1
    if size <= 0:
        print("Empty Heap")
        return None
    self.heap[self.peak] = self.heap[self.last]
    self.last -= 1
    self.max_heapify(self.peak)

```

- Step 1: Remove the Root



```
  /  \  
40 20
```

Delete: 60

- Step 2: Move the Last Node

```
    20  
   /  \  
  50   35  
 /  
40
```

The heap property is now broken.

- Step 3: Heapify Down

```
    20  
   /  \  
  50   35  
 /  
40  
  
    ↓  
    50  
   /  \  
  20   35  
 /  
40  
  
    ↓  
    50  
   /  \  
  40   35  
 /  
20
```

The heap becomes valid again. Time Complexity: $O(\log n)$

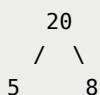
2.9. Converting a Max Heap into a Min Heap

A Min Heap follows the opposite rule. Instead of: $Parent \geq Child$, it requires: $Parent \leq Child$. To achieve this, we replace the Max Heapify operation with a Min Heapify operation.

```
def min_heapify(self, pos): # Heapify Min Heap
    if not self.is_leaf(pos):
        # In Min Heap, the child node is higher or equal
        to the father node
        if ((self.heap[pos] > self.heap[self.left_child
            (pos)])
            or (self.right_child(pos) <= self.last
                and self.heap[pos] > self.heap[self.
            right_child(pos)])):
            # Heapify the right node and recursive right
            child tree
            if (self.right_child(pos) <= self.last
                and self.heap[self.right_child(pos)]
            < self.heap[self.left_child(pos)]):
                self.swap(pos, self.right_child(pos))
                self.min_heapify(self.right_child(pos))
            else: # Heapify the left node
                self.swap(pos, self.left_child(pos))
                self.min_heapify(self.left_child(pos))
```

This function compares a parent node with its children and swaps with the smaller child whenever necessary.

Example:



Violation: $20 > 5$. After heapify, the smaller value moves upward.



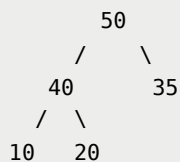
2.10. Converting an Entire Heap

The conversion process begins from the last internal node and moves upward toward the root.

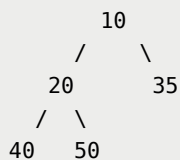
```
def convert_min_heap(self):
    last_node = (self.last - 1) // 2
    # Range from last node to the last index, down to -1
    for i in range(last_node, -1, -1):
        self.min_heapify(i)
```

Visualization:

Original Max Heap



After conversion:



The heap now satisfies the Min Heap property.

This approach is often called **Bottom-Up Heap Construction**, since heapify starts from the lowest internal nodes and gradually works upward until the root is processed.

Below is the full implementation of the Heap class:

heap.py

Key Functions to Remember

When implementing heaps, only a few functions perform most of the work:

Function	Purpose
<code>parent()</code>	Locate parent node
<code>left_child()</code>	Locate left child
<code>right_child()</code>	Locate right child
<code>insert()</code>	Add a new element
<code>delete()</code>	Remove the root element
<code>max_heapify()</code>	Restore Max Heap property
<code>min_heapify()</code>	Restore Min Heap property
<code>get_peak()</code>	Retrieve root element
<code>convert_min_heap()</code>	Convert Max Heap into Min Heap

Together, these functions allow heaps to efficiently support Priority Queues, Heap Sort, scheduling systems, graph algorithms, and many other real-world applications.

Summary

This section covers the most common operations performed on a heap, including traversal, insertion, deletion, and heapification. We learned that insertion follows the level-order structure of a complete binary tree and is followed by a heapify-up process to restore the heap property. Similarly, deletion removes the root node, replaces it with the last node in the heap, and uses heapify-down to maintain the correct ordering.

We also explored the deletion of nodes at arbitrary positions, which requires replacing the selected node with the last node and performing heapification when necessary. Through these operations, heaps can efficiently maintain their structure while supporting insertion and deletion in **$O(\log n)$** time.

Finally, we examined how these operations are implemented in Python using an array-based representation of heaps. By utilizing helper functions such as parent-child indexing and heapify procedures, both Max Heaps and Min Heaps can be managed efficiently, providing the foundation for Priority Queues and many other real-world applications.

References

https://drive.google.com/file/d/1VfNgqFPkvnDGZeVKN_QqIHjWCDQRapBM/view?usp=drive_link

<https://www.geeksforgeeks.org/dsa/difference-between-min-heap-and-max-heap/>

<https://www.codecademy.com/article/max-heap>

<https://www.youtube.com/watch?v=HqPJF2L5h9U&t=2141s>

Next Section:

 [12.3. Heap Sort](#)